Computer Communications 154 (2020) 528–533

Contents lists available at ScienceDirect

Computer Communications

journal homepage: www.elsevier.com/locate/comcom

Cooperative comodule discovery for swarm-intelligent drone arrays

Hsin Chuang [a], Kuan-Lin Hou [a], Seungmin Rho [b], Bo-Wei Chen [a,*]

[a] Department of Electrical Engineering, National Sun Yat-Sen University, Taiwan
[b] Department of Software, Sejong University, Republic of Korea

ARTICLE INFO

Keywords:
Graph signal processing
Cooperative mode
Comodule
Swarm intelligence
Self-organizing map
Sparse network factorization
Nonnegative matrix factorization

ABSTRACT

This study presents a method for discovering comodules from a drone array. Herein, a comodule represents a frequent connection pattern among drones and their connected targets (e.g., mobile phones and end users) — Frequent connections among drone-to-drone (D2D) and drone-to-target (D2T) patterns. In the Internet of Drones (IoD), where sensing and communications become a major task, it is important to explore the cooperative components (i.e., subgraphs or comodules) of the network because data streams frequently flow through those components. Once a comodule is found, the control center can notify the drones along with their connected targets to process data in a cooperative mode. However, it is difficult to extract comodules since the topology of the IoD is fluid. Additionally, the targets connected to the IoD could be dynamic and mobile. To effectively discover comodules while considering mobile targets at the same time, this study proposes regularized self-organizing sparse network factorization. It jointly decomposes a connection network formed by a drone array and its connected targets into basis and coefficient matrices. Comodules can be explored by examining those matrices. Experiments were carried out on open Call Data Records and a drone array. The results show that the proposed method generated less divergence and higher tightness between the nodes of a comodule than the baseline did.

1. Introduction

The Internet of Drones (IoD) [1,2] is known for its efficiency and mobility during deployment. Therefore, it has been widely used in scouting and sensing. However, recent demands for ultrahigh-definition data in the Visual Internet of Things (VIoT) [3] and Edge Artificial Intelligence (Edge AI) [4] has challenged the IoD. Firstly, detection and model learning for ultrahigh-definition data require intensive computation. To handle computation problems, task/network partitioning is often required because several devices can cooperate together to finish a complicated task [5,6]. Besides, different parts of the IoD can deal with various missions and share harvested information. Therefore, how to perform task/network partitioning in the IoD is important. Secondly, the IoD needs to decide where to store data and retrieve caches without creating too much overhead during transmissions [7]. Under such circumstances, frequent connection paths (i.e., subgraph patterns) in the IoD provide a useful clue. Thirdly, sensing targets (e.g., end users) may not stay fixed in the same place all the time. They may change locations with time. This implies that the IoD should be capable of adapting to environmental changes to maintain sensing stability. This requires an intelligent and automatic self-reconfiguring capability. Fourthly, when the topology is modified, the role of each drone might change. As mentioned earlier, not every drone of the IoD undertakes the same mission. For instance, some of the drones are capturing infrared thermographs, some are collecting telecommunication signals, and other drones work as sink nodes. In such a case, frequent connection paths not only need to reveal the interconnections among drones but also reflect the linkages between drones and targets. This is because the linkages between drones and targets may change. Nonetheless, current network partitioning usually focused on clustering on network nodes. Lastly, when computational resources become insufficient or sensing requests rise, the members of the IoD should be dynamically increased. The IoD topology must respond to environmental dynamics, e.g., connections from targets (see Figs. 1 and 2).

In view of such challenges, this work develops a swarm intelligence-enabled drone array, where a self-organizing map (SOM) is used to provide a self-reconfiguring capability that automatically changes the topology of the IoD. Additionally, to extract comodules from both the topology of the IoD and the connected target activities, regularized self-organizing sparse network factorization is devised in this study for discovering frequent connection paths. The SOM has several merits. The first one is that when targets relocate, the IoD can respond to the movement of targets by modifying the topology of deployment in order to maintain coverage. Moreover, if the density of target requests becomes higher/lower, the SOM supports adding or removing drones without dramatically adjusting its topology. These advantages maintain the stability of data acquisition. Besides, the role of each drone keeps

* Corresponding author.
E-mail address: bo-wei.chen@mail.nsysu.edu.tw (B.-W. Chen).

https://doi.org/10.1016/j.comcom.2020.02.077
Received 1 December 2019; Received in revised form 29 January 2020; Accepted 25 February 2020
Available online 3 March 2020
0140-3664/© 2020 Elsevier B.V. All rights reserved.



Fig. 1. Illustration of the IoD that consists of an 8 × 8 drone array.

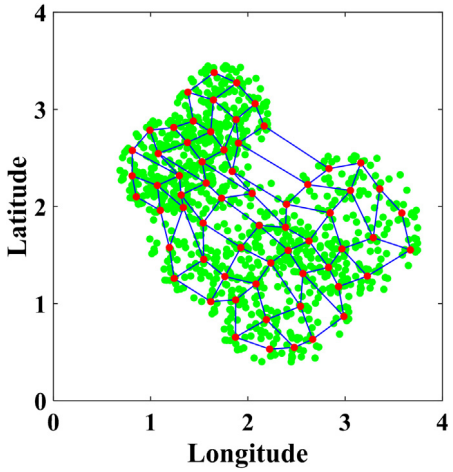


Fig. 2. Illustration of the IoD in Fig. 1 powered by swarm intelligence. Red dots represent drones, and green dots signify targets connected to drones. Edges denote connections. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

relatively stable. More importantly, the linkages between drones and targets are captured and recorded in the hitmap of the SOM. The hitmap of SOMs refers to the mapping from targets onto drones. One can utilize the SOM along with its hitmap to find comodules. A comodule reveals a frequent connection pattern among drones and their connected targets. In other words, a comodule shows a frequent connection among drone-to-drone (D2D) and drone-to-target (D2T) patterns as shown in Fig. 3.

In addition to frequent connection patterns, comodules can also discover interactions between two heterogeneous networks by displaying bipartite graphs. This is because comodules can be viewed as basis components of a graph, or equivalently basis subgraphs that preserve important structures. Such structures are conducive to task/network partitioning. Thus, it is critical to examine which targets transmit data into the IoD most or which parts of the IoD frequently receive data from targets. Besides, a comodule provides information for selecting a group of cooperative drones that work together to complete a data processing mission. Hotspots that rely on computation, data caching, and transmissions can be detected by examining comodules. Control centers are therefore capable of making decisions to choose which drones require additional supports or which paths need optimization.

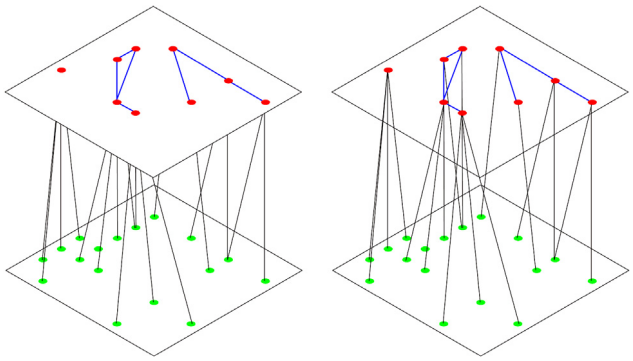


Fig. 3. Examples of comodules in the IoD. Red dots represent drones, and green dots signify targets connected to drones. Edges denote connections. Blue lines show the linkages between drones, and the linkages among drones and targets are black lines. The left and the right subfigures are the same. The right subfigure uses the transparent layout to illustrate the linkages. In this figure, there are three comodules. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

The rest of this paper is organized as follows. Section 2 details related works. Section 3 then describes the proposed method. Next, analytic results are discussed in Section 4. Conclusions are finally drawn in Section.

2. Related work

In recent years, manifold learning via graph embedded structures has become popular as it reserves local invariance. Local invariance maintains the relative relations between samples when samples are converted from data space to another type of space, e.g., low-dimensional space [8]. In this study, nonnegative matrix factorization (NMF) is highlighted due to its capability to perform graph embedding, compact representation, and clustering. One of various NMF approaches is nonnegative matrix factorization for graphs, and graph regularized nonnegative matrix factorization (GNMF) [9] is a good example to illustrate local invariance. For example, assume that there are three samples, and the first one is similar to the second one but different from the third. When those data are represented as nodes in a graph, where edges denote sample relations, the path between the first and the second one is closer than that between the first and the third one. In addition to data local invariance, GNMF can encode graph structures by using compact representation. GNMF has developed many variants, such as [10–16]. Different types of matrix norm and regularization were intensively studied in the above research. For instance, [12] focused on nuclear norm, and [11] concentrated on both nuclear norm and $\mathcal{L}_1$-norm. [16] devised correntropy norm [17] and applied it to hypergraphs. In addition to norm and regularization, [10,15] were also aimed at cofactorization for introducing more than one data source. At present, many matrix cofactorization techniques have been developed, for instance, [18–20]. Interestingly, Zhang et al. [10] proposed a method for cofactorizing heterogeneous data matrices and applied the result to comodule discovery, where two subgraph patterns were jointly extracted. Comodule discovery is derived from module analysis that is often referred to as clustering analysis in machine learning and statistics for decades. In the network/communications research fields, module analysis is related to network segmentation or graph partitioning because network modules should consider the connections while examining node similarities. Sparse Network-Regularized Multiple NMF (SNMNMF) [10] successfully integrated graph-embedded manifold learning, graph partitioning, and nonnegative matrix cofactorization together. The proposed work advances SNMNMF [10] by further adding versatile sparsity control between $\mathcal{L}_1$-norm and $\mathcal{L}_2$-norm [21] into graph regularized nonnegative matrix factorization. Sparsity based on $\mathcal{L}_1$-norm and squared $\mathcal{L}_1$-norm are also modeled in this study.

3. Proposed method

This section details comodule discovery for swarm-intelligent drone arrays. This section consists of three subsections, of which the first one is swarm intelligence powered by SOMs, and the subsequent two subsections involve SOM decomposition.

3.1. Self-organizing maps and hitmaps

The SOM is a type of neural network, which typically consists of two layers — The bottom and the top layer. The bottom layer is the input layer for receiving input data, e.g., target requests or remote sensing data. The top layer is a network (i.e., a node array) with nodes adjacent to their nearest neighbors. In this study, the nodes represent the drones of the IoD. During the self-organizing stage, the top layer responds to the iterative stimuli from the bottom layer, and the topology of the top layer gradually morphs. At present, variants of SOMs have been developed, e.g., growing SOMs, growing hierarchical SOMs, spatial SOMs, geographic SOMs, and multilayered SOMs.

Assume that after the self-organizing stage is completed, a typical SOM formed by N drones is generated. Let $\mathbf{X}_\mathrm{T}$ and $\mathbf{X}_\mathrm{S}$ denote M-by-L and M-by-N matrices respectively, where M is the dimensionality (i.e., the number of feature dimensions), and L means the number of targets. One can use matrix $\mathbf{X}_\mathrm{S}$ to represent the weights of the SOM and $\mathbf{X}_\mathrm{T}$ to signify the sensing data (i.e., a matrix) from targets. In this study, targets could be referred to as end users. This means that end users can generate data with the use of terminal devices like mobile phones or Internet-of-Things gadgets.

After the self-organizing phase finishes, a hitmap corresponding to the SOM is generated. Such a hitmap collects the number of times when an input (i.e., a target) is assigned to a node or equivalently hits a node of the SOM. Moreover, as a SOM requires iterations to complete self-organizing, each iteration generates a hitmap. If it takes C iterations for a SOM to complete the self-organizing phase, the number of times for all the inputs to hit the nodes is CN in total. Under such a circumstance, an input may hit more than one node. To display such a relational interaction between the inputs and the nodes in the hitmap, an N-by-L bipartite graph $\mathbf{R}$ is used.

In addition to hitmaps, a SOM can also be represented as a weighted adjacent matrix $\mathbf{A}$ when the self-organizing stage completes. The size of such an adjacent matrix is N-by-N. Furthermore, an element a_{ij} of $\mathbf{A}$ is zero if nodes i and j ($i \neq j$) have no connection. Otherwise, a_{ij} is the Euclidean distance between nodes i and j. Besides, when the distance is too long or larger than a threshold, a_{ij} is set to a zero. In the scenario of communications, the distance is replaced by the amount data flows.

In summary, the SOM processing stage generates four variables — $\mathbf{A}$, $\mathbf{R}$, $\mathbf{X}_\mathrm{S}$ and $\mathbf{X}_\mathrm{T}$. Those four variables are subsequently used in SOM decomposition. This study devises two types of decomposition. One is versatile sparse SOM decomposition, and the other is versatile $\mathcal{L}_1$-norm SOM decomposition. They have various degrees of sparsity. Herein, "versatile" is a terminology coined by Li and Ngom [21] in their work to describe the linear combination of $\mathcal{L}_1$-norm and $\mathcal{L}_1$-norm regularization. This study uses such a terminology to provide flexibility for controlling cofactor $\mathbf{U}$.

3.2. Versatile sparse decomposition

Sparse decomposition is aimed at finding three nonnegative matrix factors — One cluster centroid factor $\mathbf{U}$, and two coefficient factors $\mathbf{V}_\mathrm{S}$ and $\mathbf{V}_\mathrm{T}$, such that $\mathbf{X}_\mathrm{S} = \mathbf{U}\mathbf{V}_\mathrm{S}$ and $\mathbf{X}_\mathrm{T} = \mathbf{U}\mathbf{V}_\mathrm{T}$. Two matrices $\mathbf{X}_\mathrm{S}$ and $\mathbf{X}_\mathrm{T}$ share the same cofactor $\mathbf{U}$. Assume that there are D clusters after decomposition. The sizes of $\mathbf{U}$, $\mathbf{V}_\mathrm{S}$, and $\mathbf{V}_\mathrm{T}$ are respectively $M \times D$, $D \times N$, and $D \times L$. Each column of $\mathbf{U}$ represents a cluster center, and the number of columns of $\mathbf{U}$ indicates the number of clusters. For the coefficient factors, each column denotes the membership degree (i.e., weight) of a sample. The number of columns of a coefficient factor signifies the number of samples. That is, the number of columns of $\mathbf{V}_\mathrm{S}$ is the same as the total number of drones, and that of $\mathbf{V}_\mathrm{T}$ is equivalent to the total number of targets. It is worth noting that the number of rows of a coefficient factor is exactly the same as the number of clusters. Therefore, the intuition behind a coefficient factor is that when a sample (i.e., a column vector) is selected from the coefficient factor, its membership degree for each cluster is revealed in each row. In brief, the D rows of a coefficient factor show the tendency towards the D clusters.

Based on the above-mentioned concept, a node weight of a SOM or a data sample from targets is regarded as a weighted linear combination of cluster centroids. Accordingly, sparse SOM decomposition minimizes the following objective function.

$$\begin{aligned}\underset{\mathbf{U},\mathbf{V}_\mathrm{S},\mathbf{V}_\mathrm{T}}{\operatorname{argmin}}\ \varepsilon = \underset{\mathbf{U},\mathbf{V}_\mathrm{S},\mathbf{V}_\mathrm{T}}{\operatorname{argmin}}\ &\Big\{ \left\|\mathbf{X}_\mathrm{S} - \mathbf{U}\mathbf{V}_\mathrm{S}\right\|_\mathcal{F}^2 + \left\|\mathbf{X}_\mathrm{T} - \mathbf{U}\mathbf{V}_\mathrm{T}\right\|_\mathcal{F}^2 \\ &+ \alpha \operatorname{Tr}\left(\mathbf{V}_\mathrm{S}\mathbf{A}\mathbf{V}_\mathrm{S}^\top\right) + \beta \operatorname{Tr}\left(\mathbf{V}_\mathrm{S}\mathbf{R}\mathbf{V}_\mathrm{T}^\top\right) + \gamma \left\|\mathbf{U}\right\|_\mathcal{F}^2 + G \Big\}, \\ &\text{s.t. } \mathbf{U} \geq 0, \mathbf{V}_\mathrm{S} \geq 0, \text{ and } \mathbf{V}_\mathrm{T} \geq 0,\end{aligned} \tag{1}$$

where

$$G = \delta \sum_{d=1}^{D} \left\|\mathbf{U}(:,d)\right\|_1^2 + \lambda \left(\sum_{n=1}^{N} \left\|\mathbf{V}_\mathrm{S}(:,n)\right\|_1^2 + \sum_{l=1}^{L} \left\|\mathbf{V}_\mathrm{T}(:,l)\right\|_1^2 \right). \tag{2}$$

Meanwhile, $\mathcal{F}$ denotes the Frobenius norm, $\operatorname{Tr}(\cdot)$ computes the trace, $\|\cdot\|_1$ is $\mathcal{L}_1$ norm, and $\top$ means the transpose operator. Besides, α, β, γ, δ, and λ are predefined model parameters, where $\gamma + \delta = 1$. The last term of (1) is a sparse constraint based on [22]. Kim and Park [22] developed the squared $\mathcal{L}_1$ norm to ensure sparsity. As all the matrix factors are nonnegative, (2) can be rewritten as

$$\begin{aligned}G = \ &\delta \sum_{d=1}^{D} \left(\sum_{m=1}^{M} \mathbf{U}(m,d) \right)^2 \\ &+ \lambda \left(\sum_{n=1}^{N} \left(\sum_{d=1}^{D} \mathbf{V}_\mathrm{S}(d,n) \right)^2 + \sum_{l=1}^{L} \left(\sum_{d=1}^{D} \mathbf{V}_\mathrm{T}(d,l) \right)^2 \right).\end{aligned} \tag{3}$$

Plugging (3) into (1) yields another cost function. However, it is nonconvex and difficult to solve. By focusing only on one variable while fixing the other variables, (1) become convex, and it can be solved by using Alternating Nonnegative Least Squares. Taking the partial derivative of (1) and applying the multiplicative update rules with respect to $\mathbf{U}_{md}$, $(\mathbf{V}_\mathrm{S})_{dn}$, and $(\mathbf{V}_\mathrm{T})_{dl}$ yield

$$\mathbf{U}_{md} \leftarrow \max\left(\epsilon, \mathbf{U}_{md} \odot \frac{\left(\mathbf{X}_\mathrm{T}\mathbf{V}_\mathrm{T}^\top + \mathbf{X}_\mathrm{S}\mathbf{V}_\mathrm{S}^\top\right)_{md}}{\left(\mathbf{U}\mathbf{V}_\mathrm{S}\mathbf{V}_\mathrm{S}^\top + \mathbf{U}\mathbf{V}_\mathrm{T}\mathbf{V}_\mathrm{T}^\top + \gamma\mathbf{U} + \delta\mathbf{e}_M\mathbf{e}_M^\top\mathbf{U}\right)_{md}} \right), \tag{4}$$

$$\left(\mathbf{V}_\mathrm{S}\right)_{dn} \leftarrow \max\left(\epsilon, \left(\mathbf{V}_\mathrm{S}\right)_{dn} \odot \frac{\left(\mathbf{U}^\top\mathbf{X}_\mathrm{S}\right)_{dn}}{\left(\mathbf{U}^\top\mathbf{U}\mathbf{V}_\mathrm{S} + \alpha\mathbf{V}_\mathrm{S}\mathbf{A}^\top + \frac{1}{2}\beta\mathbf{V}_\mathrm{T}\mathbf{R}^\top + \lambda\mathbf{e}_D\mathbf{e}_D^\top\mathbf{V}_\mathrm{S}\right)_{dn}} \right), \tag{5}$$

and

$$\left(\mathbf{V}_\mathrm{T}\right)_{dl} \leftarrow \max\left(\epsilon, \left(\mathbf{V}_\mathrm{T}\right)_{dl} \odot \frac{\left(\mathbf{U}^\top\mathbf{X}_\mathrm{T}\right)_{dl}}{\left(\mathbf{U}^\top\mathbf{U}\mathbf{V}_\mathrm{T} + \frac{1}{2}\beta\mathbf{V}_\mathrm{S}\mathbf{R} + \lambda\mathbf{e}_D\mathbf{e}_D^\top\mathbf{V}_\mathrm{T}\right)_{dl}} \right), \tag{6}$$

where $\mathbf{e}_M$ and $\mathbf{e}_D$ are $M \times 1$ and $D \times 1$ column vectors with all the elements set to ones, ϵ denotes an extremely small positive value, $\odot$ is the element-wise multiplication operator, and division is also element-wise. At the beginning, $\mathbf{U}$, $\mathbf{V}_\mathrm{S}$, and $\mathbf{V}_\mathrm{T}$ are initialized, e.g., with random nonnegative values. Subsequently, (4), (5), and (6) iterate until $\mathbf{U}$, $\mathbf{V}_\mathrm{S}$, and $\mathbf{V}_\mathrm{T}$ converge respectively.

3.3. Versatile $\mathcal{L}_1$-norm decomposition

$\mathcal{L}_1$-norm decomposition differs from sparse decomposition in the constraint G. Sparse decomposition uses squared $\mathcal{L}_1$ norm, where $\mathcal{L}_1$-norm decomposition simply focuses on $\mathcal{L}_1$ norm. The objective function

is

$$\begin{aligned}\underset{\mathbf{U},\mathbf{V}_\mathrm{S},\mathbf{V}_\mathrm{T}}{\operatorname{argmin}}\ \varepsilon' = \underset{\mathbf{U},\mathbf{V}_\mathrm{S},\mathbf{V}_\mathrm{T}}{\operatorname{argmin}}\ &\Big\{ \left\|\mathbf{X}_\mathrm{S}-\mathbf{U}\mathbf{V}_\mathrm{S}\right\|_\mathcal{F}^2 + \left\|\mathbf{X}_\mathrm{T}-\mathbf{U}\mathbf{V}_\mathrm{T}\right\|_\mathcal{F}^2 \\ &+ \alpha \operatorname{Tr}\left(\mathbf{V}_\mathrm{S}\mathbf{A}\mathbf{V}_\mathrm{S}^\top\right) + \beta \operatorname{Tr}\left(\mathbf{V}_\mathrm{S}\mathbf{R}\mathbf{V}_\mathrm{T}^\top\right) + \gamma \left\|\mathbf{U}\right\|_\mathcal{F}^2 + H \Big\}, \\ &\text{s.t. } \mathbf{U}\geq 0, \mathbf{V}_\mathrm{S}\geq 0, \text{and } \mathbf{V}_\mathrm{T}\geq 0,\end{aligned} \tag{7}$$

where

$$H = \delta \sum_{d=1}^{D} \left\|\mathbf{U}(:,d)\right\|_1 + \lambda\left(\sum_{n}^{N}\left\|\mathbf{V}_\mathrm{S}(:,n)\right\|_1 + \sum_{l}^{L}\left\|\mathbf{V}_\mathrm{T}(:,l)\right\|_1\right). \tag{8}$$

Taking the partial derivative of (7) and converting the results to the multiplicative update rules with respect to $\mathbf{U}_{md}$, $(\mathbf{V}_\mathrm{S})_{dn}$, and $(\mathbf{V}_\mathrm{T})_{dl}$ yield

$$\mathbf{U}_{md} \leftarrow \max\left(\epsilon, \mathbf{U}_{md}\odot \frac{\left(\mathbf{X}_\mathrm{T}\mathbf{V}_\mathrm{T}^\top + \mathbf{X}_\mathrm{S}\mathbf{V}_\mathrm{S}^\top\right)_{md}}{\left(\mathbf{U}\mathbf{V}_\mathrm{S}\mathbf{V}_\mathrm{S}^\top + \mathbf{U}\mathbf{V}_\mathrm{T}\mathbf{V}_\mathrm{T}^\top + \gamma\mathbf{U} + \frac{1}{2}\delta\mathbf{E}\right)_{md}}\right), \tag{9}$$

$$\left(\mathbf{V}_\mathrm{S}\right)_{dn} \leftarrow \max\left(\epsilon, \left(\mathbf{V}_\mathrm{S}\right)_{dn}\odot \frac{\left(\mathbf{U}^\top\mathbf{X}_\mathrm{S}\right)_{dn}}{\left(\mathbf{U}^\top\mathbf{U}\mathbf{V}_\mathrm{S} + \alpha\mathbf{V}_\mathrm{S}\mathbf{A}^\top + \frac{1}{2}\beta\mathbf{V}_\mathrm{T}\mathbf{R}^\top + \frac{1}{2}\lambda\mathbf{E}_\mathrm{S}\right)_{dn}}\right), \tag{10}$$

and

$$\left(\mathbf{V}_\mathrm{T}\right)_{dl} \leftarrow \max\left(\epsilon, \left(\mathbf{V}_\mathrm{T}\right)_{dl}\odot \frac{\left(\mathbf{U}^\top\mathbf{X}_\mathrm{T}\right)_{dl}}{\left(\mathbf{U}^\top\mathbf{U}\mathbf{V}_\mathrm{T} + \frac{1}{2}\beta\mathbf{V}_\mathrm{S}\mathbf{R} + \frac{1}{2}\lambda\mathbf{E}_\mathrm{T}\right)_{dl}}\right), \tag{11}$$

where $\mathbf{E}$, $\mathbf{E}_\mathrm{S}$, and $\mathbf{E}_\mathrm{T}$ are $M\times D$, $D\times N$, and $D\times L$ matrices with all the elements equal to ones. To obtain the solution to (7), the algorithm for Alternating Nonnegative Least Squares is used by repeating computing (9), (10), and (11) until $\mathbf{U}$, $\mathbf{V}_\mathrm{S}$, and $\mathbf{V}_\mathrm{T}$ converge respectively.

3.4. Cooperative comodule discovery

To discover comodules, this study follows the concept by [10]. The idea is straightforward. For example, when a drone is selected, one can locate its index by checking its corresponding column vector in $\mathbf{V}_\mathrm{S}$. As each row of the column vector in $\mathbf{V}_\mathrm{S}$ represents the membership degree to each cluster, one can use a threshold to determine the assignment of the drone to clusters. Likewise, given a target, one can find out its cluster membership by examining $\mathbf{V}_\mathrm{T}$. It is worth noting that a drone or a target may exist in more than one cluster, depending on the threshold. When all the drones and targets are scanned, the system can extract the members of a cluster and create a graph with the number of vertices equal to that of members. The edges of the graph are built based on the relation defined in $\mathbf{A}$ and $\mathbf{R}$. No edges are connected across clusters. When comodule discovery is done, there are D comodules. The following steps list the algorithm for comodule discovery

Algorithm: Comodule Discovery

Input: $\mathbf{X}_\mathrm{S}$, $\mathbf{X}_\mathrm{T}$, $\mathbf{A}$, and $\mathbf{R}$
Output: D graphs (i.e., D comodules)
For drone $n = 1{:}N$,
  For cluster $d = 1{:}D$,
    If $\mathbf{V}_\mathrm{S}(d,n) > \theta_\mathrm{S}$
      assign drone n to cluster d
For target $l = 1{:}L$,
  For cluster $d = 1{:}D$,
    If $\mathbf{V}_\mathrm{T}(d,n) > \theta_\mathrm{T}$
      assign target l to cluster d
For cluster $d = 1{:}D$,
  list all the drones and targets in cluster d
  build an empty graph d
  build graph vertices based on the drones & targets
  build graph edges based on $\mathbf{A}$ and $\mathbf{R}$

4. Experimental result

Experiments on open "Call Data Records (CDRs)" (dandelion.eu) were conducted. Random 1000 records were selected from the dataset, and the coordinates of the CDRs were used as data features. All the records were normalized and scaled to nonnegative values. The size of the drone array was 16 × 16. Total iterations for SOMs were 3000, and two hitmaps were extracted. Model parameters α, β, γ, δ, and λ were empirically set to 0.50. The number of testing comodules ranged from three to nine, with a separation of two. After the self-organizing stage, a converged SOM was formed. Then, it was converted into an adjacency matrix, where the entries represented the normalized Euclidean distance between drones. Those entries larger than 0.05 were removed.

For SOM decomposition, three models were examined in the experiments. One is the method by Zhang et al. [10], another is the proposed sparse SOM decomposition, and the other is the proposed $\mathcal{L}_1$-norm SOM decomposition. All the three algorithms used 0.30 for thresholds θ_S and θ_T. After decomposition process was finished, this study conducted the following two types of measures for evaluating the quality of comodules. One is divergence $\mathcal{D}$, and the other is tightness $\mathcal{T}$. First, as a comodule consists of heterogeneous vertices, i.e., drones and targets, the divergence between the means of these two groups, was computed. It is worth noting that the self-organizing process of SOMs is like assimilation, where SOM nodes and target data with similar features gather together. Thus, if the divergence between two groups in a comodule is small, it reveals that an algorithm can select good qualities of comodules. The equation $\mathcal{D} = \left\|\frac{1}{M}\left(\boldsymbol{\mu}_\mathrm{S} - \boldsymbol{\mu}_\mathrm{T}\right)\right\|_2$ shows the divergence, where $\boldsymbol{\mu}_\mathrm{S}$ and $\boldsymbol{\mu}_\mathrm{T}$ are the means of these two groups, i.e., the SOM and the targets. Secondly, comodules are actually bipartite graphs, but the first measure "divergence" does not consider the connections. To solve such a problem, this study examined the tightness between two connected vertices, with each other on the different sides of the same bipartite graph, respectively. In another word, the connections on the same side of a bipartite graph were not computed. The equation of tightness is expressed as $\mathcal{T} = \sum w_{p,q}/N_\mathrm{R}$, where $w_{p,q}$ denotes the weight of a cut edge in a bipartite graph $\mathbf{R}$, p is a SOM node, and q is a target. Furthermore, $w_{p,q}$ is computed based on the Euclidean distance between p and q, and N_R stands for the number of cut edges. Since tightness represents the distance between p and q, they become more similar when the distance is shorter.

Experimental results based on – Divergence between two groups in a comodule – are shown in Fig. 4. The horizontal axis denotes the different number of comodules D, and the vertical axis represents the measurements. Meanwhile, "I" shapes denote standard deviation, and the mean of the measurement is shown in the center of I-shapes. As shown in the figure, observations showed that the proposed Versatile Sparse Decomposition (VSD) was better than the baseline (i.e., SNMNMF) when the number of comodules was set to 5–9. The numeric result is listed in Table 1. As a whole, the proposed Versatile Sparse Decomposition generated lower divergence than the baseline and the proposed Versatile $\mathcal{L}_1$-Norm Decomposition (VLD).

For the tightness between two connected vertices, Fig. 5 and Table 2 display the experimental results. Compared with divergence, examinations revealed that the proposed Versatile $\mathcal{L}_1$-Norm Decomposition was better than the baseline and the proposed Versatile Sparse Decomposition. Based on the experimental results, the proposed technique that used versatile controls over matrix factors indeed created positive effects on discovery of comodules. Fig. 6 illustrates an example of comodules, where red dots represent drones, and green dots signify targets connected to drones. Furthermore, red edges denote inter-drone connections, and green edges are drone-target connections. The proposed system can extract frequent D2D and D2T patterns.

Table 1
Divergence of comodules.

Algorithm	Number of comodules			
	3	5	7	9
SNMNMF	0.016767 ± 0.097185	0.310887 ± 0.337127	0.297398 ± 0.357379	0.094147 ± 0.230386
Proposed Versatile Sparse Decomposition	0.015854 ± 0.012359	0.022698 ± 0.069930	0.026391 ± 0.097385	0.032530 ± 0.114666
Proposed Versatile $\mathcal{L}_1$-Norm Decomposition	0.018060 ± 0.018341	0.054235 ± 0.154083	0.048895 ± 0.138813	0.038891 ± 0.129741

Table 2
Tightness of comodules.

Algorithm	Number of comodules			
	3	5	7	9
SNMNMF	0.000022 ± 0.000026	0.000011 ± 0.000011	0.000022 ± 0.000023	0.000024 ± 0.000028
Proposed Versatile Sparse Decomposition	0.000013 ± 0.000011	0.000012 ± 0.000011	0.000012 ± 0.000010	0.000011 ± 0.000010
Proposed Versatile $\mathcal{L}_1$-Norm Decomposition	0.000014 ± 0.000014	0.000010 ± 0.000011	0.000010 ± 0.000012	0.000010 ± 0.000011

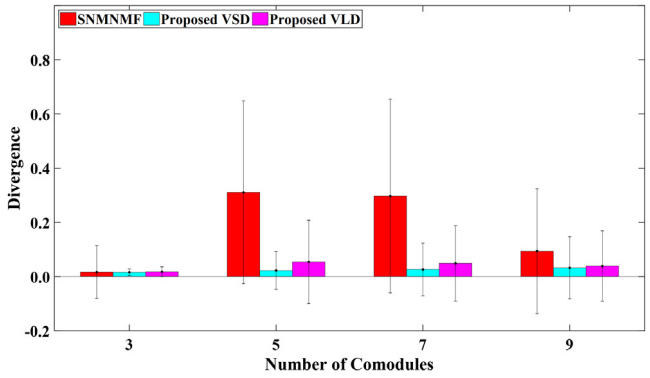


Fig. 4. Divergence between two groups in a comodule.

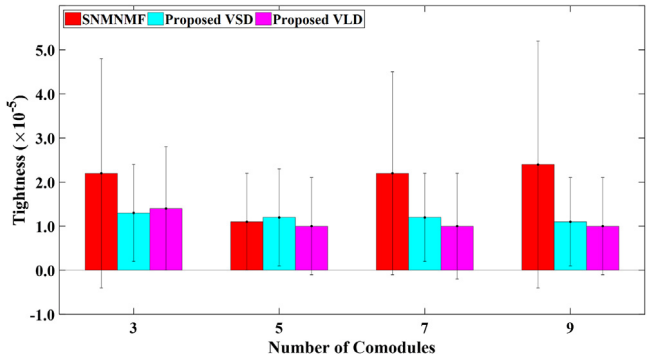


Fig. 5. Tightness between two connected vertices.

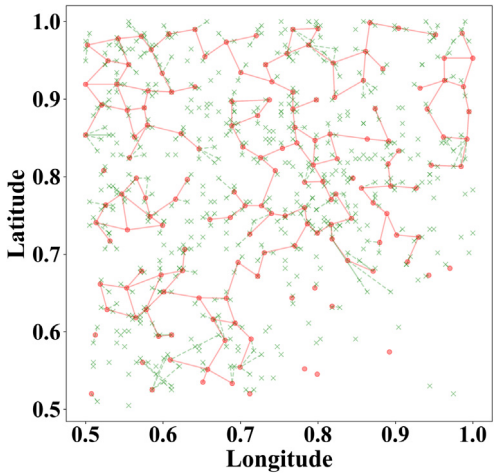


Fig. 6. Example of comodules generated based on CDRs.

5. Conclusion

This study presents a novel SOM decomposition with versatile sparsity controls for discovering comodules in the IoD. Two types of versatile sparsity constraints — $\mathcal{L}_1$-norm and squared $\mathcal{L}_1$-norm — are proposed in the study. These two types of controls offer flexibility to control cofactor **U**. For the IoD, it is important to find comodules because comodules reveal frequent connections patterns. This allows the dispatch/control center to understand the hotspots of the deployment. The proposed method enhances the typical IoD by providing a swarm-intelligent drone array powered by SOMs, which can self-adapt to environmental changes. Furthermore, after topological morphing, the proposed method can discover comodules from a fluid SOM.

Experiments were conducted on an open dataset. The results showed that the proposed method generated higher tightness between the connections of bipartite graphs and lower divergence between targets and drones, better than the baseline. This verified the effectiveness of the proposed method.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CRediT authorship contribution statement

Kuan-Lin Hou: Software. **Bo-Wei Chen:** Supervision, Funding acquisition.

Acknowledgments

This work is supported in part by the Ministry of Science and Technology, Taiwan (107-2218-E-110-013-MY3) and in part by 2019 Nvidia GPU program. The authors would like to thank Freepik for contributing the icons in Fig. 3.

References

[1] E.K. Wang, C.-M. Chen, F. Wang, M.K. Khan, S. Kumari, Joint-learning segmentation in Internet of drones (IoD)-based monitor systems, Comput. Commun. 152 (2020) 54–62.

[2] F. Al-Turjman, M. Abujubbeh, A. Malekloo, L. Mostarda, UAVs assessment in software-defined IoT networks: An overview, Comput. Commun. 150 (2020) 519–536.

[3] W. Ji, J. Xu, H. Qiao, M. Zhou, B. Liang, Visual IoT: Enabling Internet of Things visualization in smart cities, IEEE Netw. 33 (2) (2019) 102–110.
[4] Z. Zhou, X. Chen, E. Li, L. Zeng, K. Luo, J. Zhang, Edge Intelligence: Paving the last mile of artificial intelligence with edge computing, Proc. IEEE 107 (8) (2019) 1738–1762.
[5] W.-C. Chien, H.-Y. Weng, C.-F. Lai, Q-learning based collaborative cache allocation in mobile edge computing, Future Gener. Comput. Syst. 102 (2020) 603–610.
[6] G. Wu, Y. Miao, Y. Zhang, A. Barnawi, Energy efficient for UAV-enabled mobile edge computing networks: Intelligent task prediction and offloading, Comput. Commun. 150 (2020) 556–562.
[7] R. Wang, M. Li, L. Peng, Y. Hu, M.M. Hassan, A. Alelaiwi, Cognitive multi-agent empowering mobile edge computing for resource caching and collaboration, Future Gener. Comput. Syst. 102 (2020) 66–74.
[8] J.-X. Liu, D. Wang, Y.-L. Gao, C.-H. Zheng, Y. Xu, J. Yu, Regularized nonnegative matrix factorization for identifying differentially expressed genes and clustering samples: A survey, IEEE/ACM Trans. Comput. Biol. Bioinform. 15 (3) (2018) 974–987.
[9] D. Cai, X. He, J. Han, T.S. Huang, Graph regularized nonnegative matrix factorization for data representation, IEEE Trans. Pattern Anal. Mach. Intell. 33 (8) (2011) 1548–1560.
[10] S. Zhang, Q. Li, J. Liu, X.J. Zhou, A novel computational framework for simultaneous integration of multiple types of genomic data to identify microRNA-gene regulatory modules, Bioinformatics 27 (13) (2011) i401–i409.
[11] S. Yang, C. Hou, C. Zhang, Y. Wu, S. Weng, Robust non-negative matrix factorization via joint sparse and graph regularization, in: Proc. 2013 International Joint Conference on Neural Networks, Dallas, Texas, United States, 2013, Aug. 04–09, pp. 1–5.
[12] J. Huang, F. Nie, H. Huang, C. Ding, Robust manifold nonnegative matrix factorization, ACM Trans. Knowl. Discov. Data 8 (3) (2014) 1–21.
[13] Y. Pei, N. Chakraborty, K. Sycara, Nonnegative matrix tri-factorization with graph regularization for community detection in social networks, in: 24th International Conference on Artificial Intelligence Buenos Aires, Argentina, 2015, pp. 2083–2089.
[14] C. Wang, X. Song, J. Zhang, Graph regularized nonnegative matrix factorization with sample diversity for image representation, Eng. Appl. Artif. Intell. 68 (2018) 32–39.
[15] Y. Xie, W. Zhang, Y. Qu, L. Dai, D. Tao, Hyper-Laplacian regularized multilinear multiview self-representations for clustering and semisupervised learning, IEEE Trans. Cybern. (2018) 1–15.
[16] T. Jin, R. Ji, Y. Gao, X. Sun, X. Zhao, D. Tao, Correntropy-induced robust low-rank hypergraph, IEEE Trans. Image Process. 28 (6) (2019) 2755–2769.
[17] Y. Zhang, Z. Sun, R. He, T. Tan, Robust low-rank representation via correntropy, in: Proc. 2013 2nd IAPR Asian Conference on Pattern Recognition, Naha, Japan, 2013, Nov. 05–08, pp. 461–465.
[18] J. Yoo, S. Choi, Matrix co-factorization on compressed sensing, in: Proc. 21st International Joint Conference on Artificial Intelligence, Barcelona, Catalonia, Spain, 2011, Jul. 16–22, pp. 1595–1602.
[19] N. Seichepine, S. Essid, C. Févotte, O. Cappé, Soft nonnegative matrix co-factorization, IEEE Trans. Signal Process. 62 (22) (2014) 5940–5949.
[20] L. Zhang, S. Zhang, A general joint matrix factorization framework for data integration and its systematic algorithmic exploration, IEEE Trans. Fuzzy Syst. (2019).
[21] Y. Li, A. Ngom, Versatile sparse matrix factorization and its applications in high-dimensional biological data analysis, in: Proc. IAPR International Conference on Pattern Recognition in Bioinformatics, Nice, France, 2013, Jun. 17–20, pp. 91–101.
[22] H. Kim, H. Park, Nonnegative matrix factorization based on alternating nonnegativity constrained least squares and active set method, SIAM J. Matrix Anal. Appl. 30 (2) (2008) 713–730.